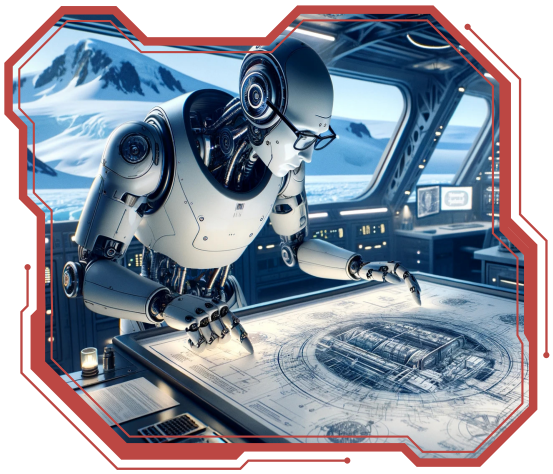
Session 4: Keeping Track of Complex Analyses

Workflow management using snakemake

Kosmas Hench

2024-03-08

Let's return to our reviewer wondering about "Figure 2". Suppose they managed to find the actual script producing the figure (nicely located in the `code/` *section). They are happy to find that the code is well documented, and it is even nicely tracked by version control. Yet, the script for "Figure 2" resides next to scripts for four further figures, as well as for a hole suite of supplementary figures, tables and drafts. To make matters worse, all of these seem to rely on results created by a combination of several data processing and analysis scripts. Some scripts seem to read in data from input files, some scrape data from the web, here seems to be some simulation going on, and there it looks like stuff is combined, transformed and then re-exported. Yet, where does it all start, how is it connected and which of the ~5000 lines of code need to be understood to judge if the basis of Figure 2 is sound? Once again, our poor reviewer might be lost...*



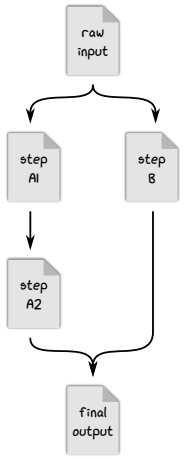
created using dall·e 3

1. A Roadmap for Your Project

Back in Session 1, we discussed how designing script such that it is ruannable from the command line increases reproducibility compared to running the code interactively within an IDE. This is because when the code is executed statically from start to end, the order in which each command is executed is pre-defined explicitly. The same issue can arise on a larger scale for a project as a whole: As soon as the project complexity rises just a little above the simplest cases, we might need a more explicit way of communicating structure of an analysis on a meta-level. Basically, what we need is a precise roadmap for the entire project, not just for its individual steps.

For a research project, the idea of a roadmap translates into a ***workflow***, which precisely describes how the individual pieces of analysis are connected. It explains the order in which individual scripts are executed, what input files they require and what output and results they create. Beyond this, it enables you to record the parameter settings and version numbers for the programs that are used throughout the analysis. Generally, the use of a workflow manager provides an additional way to document in a structured way which steps are needed to create each file of your project, which increases the ***data provenance*** of your results.

Workflow example:



2. Simply Press Play — the Beauty of Automation

Yet, while specifying a workflow for your project increases the transparency of your research, in fact the main practical benefit is that it enables the usage of workflow managers. These don't just describe the order in which the individual steps of your project need to be run, they also manage their execution. Anyone who has ever detected a bug in early script in a late stage of an analysis can probably relate to the worry of having to re-run everything downstream of this error. The beauty of using a workflow manager is that you only have to put in the effort to declare the logic of your analysis once and then these kinds of re-runs can be automatized. As a bonus, they

Data provenance describes [the] trail of methods, versions, and arguments that were used to generate a set of files. — Wratten *et al.* (2021)

enable the automatized reproduction of your analysis by others, thereby boosting the reproducibility of the project as a whole.

There are several workflow managers around (Wratten *et al.*, 2021), and in this workshop we will be focusing on `snakemake`. While this `python` based domain specific language (DSL) has a somewhat steeper learning curve than other tools (e.g. `galaxy`), it is quite flexible with respect to the workflow structure and the working environment.

3. Portability and Scalability

Similarly to how workflow managers can automate the workflow execution, they also allow to capture the working environment for each step, going beyond the mere documentation of version numbers. Within the workflow specification, it is possible to refer to well-defined working environments using `conda` environments or containerization (more on this in Session 5). This makes it possible to easily restore the used environment on a different computer and to continue or repeat the analysis in the same computational setting.

In addition, workflow managers allow to flexibly utilize the computational resources that are at hand, for example by adjusting the available memory or cores used for the execution. As they can also switch between executing jobs locally and submitting them to a computing cluster or cloud setup, projects using workflow managers can be up- or down-scaled while ensuring efficient usage of the available infrastructure.

4. Prototyping, Re-entry and Adaptability

The fact that the same code can be run in similar conditions on a regular laptop and on heavy-duty computing cluster makes prototyping and testing code for a complex analysis rather convenient. This is because it is easy to design and test an analysis pipeline locally using a tiny test-set, which allows quickly checking the general runnability of the code. Later, this can be ported to a more capable setup, where the actual analysis can be run on the full data.

Conveniently, workflow managers keep a tab on the progress of a project (usually by monitoring the files that have already been created or by logging the steps that have already been executed). It therefore is usually possible to re-enter a workflow after it has been interrupted precisely where it had stopped and prevents having to re-run everything from scratch. This is important for picking up the analysis after a particular step failed and caused the whole pipeline to break. But this feature also allows to incrementally build up a complex pipeline, extending the required steps as you progress with the analysis (and if some of your additions fail, that at least does not invalidate the results of the previous steps). Yet at the end of this you have a workflow that can be run in a single go to reproduce the entire analysis — the entire process can be very similar to the interactive development of an individual script that is then run statically.

By extension, this option to successively build on a workflow also enables adaptability for published analyses. Just like a new workflow can be extended step by step, it is rather straight forward to access and adapt an existing (published) workflow. This makes it much easier to re-use and build upon previous work, increasing the value of your published code also as a potential foundation for future work.

Recap

By specifying a workflow for your analysis, the origin of all of your results is documented, thereby ensuring **data provenance**.
In combination with the usage of a workflow manager, this often enables you to **automate the whole analysis** from acquiring the raw data to the creation of the final figures and tables of your study.
It is possible to specify the working environment alongside the code and to flexibly adjust the required resources, making workflows **portable and scalable**.
This makes it possible to prototype an analysis locally and then deploy it in a heavy-duty setup like a computing cluster or cloud service.
Finally, the option to re-enter an interrupted workflow allows to **build a workflow incrementally** as you go and to **easily adapt** previous work to your own needs.

References and Further Reading

Di Tommaso, P. *et al.* (2017). Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35:316.

Mölder, F. *et al.* (2021). Sustainable data analysis with Snakemake. Technical Report 10:33, F1000Research.

Wratten, L., Wilm, A. and Göke, J. (2021). Reproducible, scalable, and shareable analysis pipelines with bioinformatics workflow managers. *Nature Methods*, 18(10):1161–1168. DOI: 10.1038/s41592-021-01254-9.